

Red Team Agent – Lab Walkthrough

OWASP Juice Shop | Local LLM (llama3.2) | Kali Linux

Target: <http://localhost:8082>

Overview

This walkthrough documents the exact steps taken to get the `run_agent.py` red team script working end-to-end in the lab environment. Follow each step in order. Screenshots should be inserted where placeholders appear.

Step 1 – Start OWASP Juice Shop

The target application must be running on port 8082 before the scans can return meaningful results. Pull and start the Juice Shop Docker container.

```
docker run -d -p 8082:3000 --name juice-shop bkimminich/juice-shop
```

```
(kali@kali)-[~/Labfiles5.1]
$ docker run -d -p 8082:3000 --name juice-shop bkimminich/juice-shop
```

Wait approximately 30 seconds for the container to start, then verify it is up:

```
docker ps | grep juice-shop
```

```
curl http://localhost:8082
```

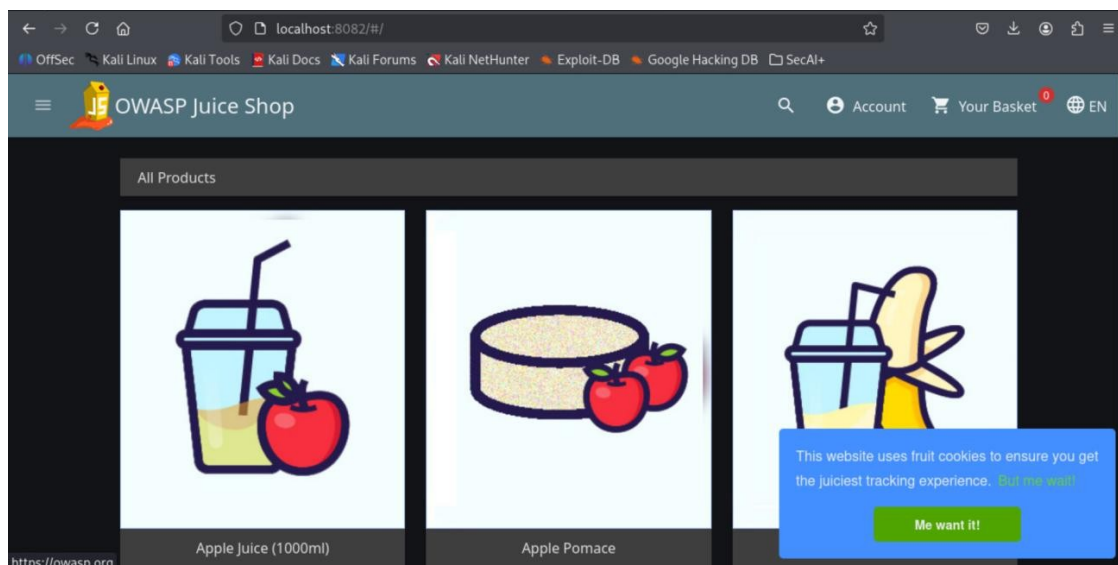
```
(kali@kali) ~/[Labfiles5.1]
└─$ curl http://localhost:8082
```

```
- Copyright (c) 2014-2026 Bjoern Kimminich & the OWASP Juice Shop contributors.
- SPDX-License-Identifier: MIT
```

```
<!doctype html>
<html lang="en" data-beasties-container>
<head>
  <meta charset="utf-8">
  <title>OWASP Juice Shop</title>
  <meta name="Description" content="Probably the most modern and sophisticated insecure web application">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="preconnect" href="https://fonts.googleapis.com/">
  <link rel="preconnect" href="https://fonts.gstatic.com/" crossorigin>
  <script>font-face{font-family:'VT323';font-style:normal;font-weight:400;font-display:swap;src:url(https://fonts.gstatic.com/s/vt323/v18/pxikyp0ihIEF2isQfJXGdg,woff2) format('woff2');unicode-range:U+0102-0103,U+0110-0111,U+0128-0129,U+0168-0169,U+01A0-01A1,U+01AF-01B0,U+0300-0301,U+0303-0304,U+0308-0309,U+0323,U+0329,U+039A-039F,U+04AB-04AF;}font-face{font-family:'VT323';font-style:normal;font-weight:400;font-display:swap;src:url(https://fonts.gstatic.com/s/vt323/v18/pxikyp0ihIEF2isRFJXGdg,woff2) format('woff2');unicode-range:U+0100-02BA,U+02C7-02C5,U+02C6-02CF,U+A720-A7FF;}font-face{font-family:'VT323';font-style:normal;font-weight:400;font-display:swap;src:url(https://fonts.gstatic.com/s/vt323/v18/pxikyp0ihIEF2isfJ3U,woff2) format('woff2');unicode-range:U+0000-00FF,U+0131,U+0152-0153,U+02BB-02BC,U+02C6,U+02DA,U+02DC,U+0304,U+0308,U+0329,U+2000-206F,U+20AC,U+2122,U+2191,U+2212,U+2215,U+FEFF}</script>
  <script>
    window.addEventListener("load",function(){
      window.cookieconsent.initialise({
        "palette": {
          "popup": { "background": "var(--theme-primary)", "text": "var(--theme-text)" },
          "button": { "background": "var(--theme-accent)", "text": "var(--theme-text)" }
        },
        "theme": "classic",
        "position": "bottom-right",
        "content": { "message": "This website uses fruit cookies to ensure you get the juiciest tracking experience.", "dismiss": "Me want it!", "link": "B ut me wait!", "href": "https://www.youtube.com/watch?v=9PNkL3wuH4" }
      });
    });
  </script>
```

If you get such HTML script response it means it is up

You can also open `http://localhost:8082` in the browser to confirm the Juice Shop UI loads.



Troubleshooting: if `docker ps | grep juice-shop` returns nothing and `curl` cannot connect, the container is not running. If you also see a "Conflict. The container name /juice-shop is already in use" error, a juice-shop container already exists from an earlier run but is stopped, so the name is taken and port 8082 has nothing listening on it.

Option A – restart the existing container (no re-download):

```
docker start juice-shop
```

Option B – remove the old container and start fresh:

```
docker rm -f juice-shop
```

```
docker run -d -p 8082:3000 --name juice-shop bkimminich/juice-shop
```

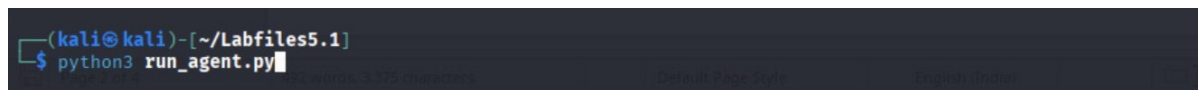
Then verify again with the two commands above – `docker ps` should now show a line mapping `0.0.0.0:8082->3000/tcp`, and `curl` should return the Juice Shop HTML.

Step 2 – Final Successful Run

With Juice Shop running and `agent_local.py` updated, run the script again. All six tools will execute against the live target and the LLM will generate a security report.

```
cd ~/Labfiles5.1
```

```
python3 run_agent.py
```



The script will print progress for each tool as it runs:

- `scan_website` – checks HTTP status and headers
- `discover_api_endpoints` – probes `/api/`, `/rest/`, `/swagger.json`, etc.
- `enumerate_directories` – checks `/admin`, `/login`, `/robots.txt`, `/ftp`, etc.
- `check_admin_endpoints` – checks admin-specific REST paths
- `juiceshop_xss_search_alert` – tests XSS via the search endpoint
- `juiceshop_sql_admin_login_bypass` – tests SQL injection on the login endpoint

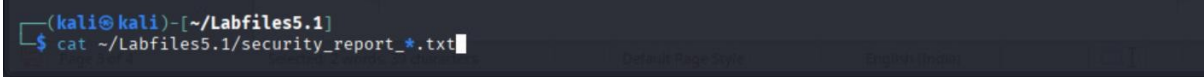
After the tools finish the LLM analyses the results and writes the report. Expected findings:

- SQLi: vulnerable = true (admin login bypass confirmed)
- XSS: payload reflected via search route
- Multiple admin endpoints returning HTTP 200

Step 3 – View the Security Report

The report is saved as security_report_TIMESTAMP.txt in the Labfiles5.1 directory. Open it to review the full findings.

```
cat ~/Labfiles5.1/security_report_*.txt
```



```
(kali㉿kali)-[~/Labfiles5.1]  
$ cat ~/Labfiles5.1/security_report_*.txt
```

The report contains: vulnerability summary, accessible paths, raw tool output, AI analysis section, and a high-level AI synopsis (3-5 bullet points suitable for training use).
